XStore

eXtreme performance in distributed kv-Store for time-series

Lan Nguyen

Data-Intensive Distributed Systems Laboratory
Illinois Institute of Technology

May 9, 2022



ILLINOIS TECH | College of Computing

1. Introduction
2. XStore
3. Evaluations
4. Next Steps

1 Introduction

2 XStore

3 Evaluations

4 Next Steps

Research Domain

Data Storage:

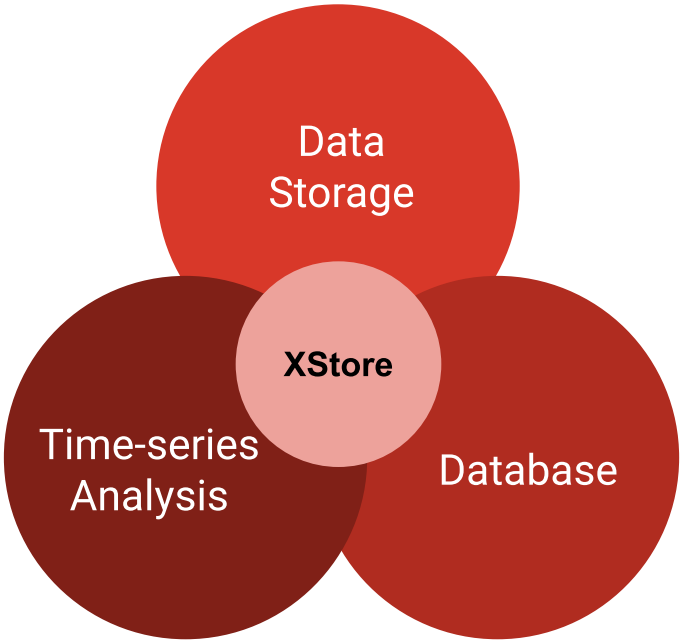
- Stores data in a semi-organized on various medium ranging from RAM, HDD and SSD
- Applications: Network file-systems, parallel file-systems and distributed file-systems

Database:

- Data storage management system designed to hold and maintain data in a pre-defined structure
- Applications: TimescaleDB, InfluxDB, MongoDB

Time-series Analysis:

- A successive sequence of data points organized by time and measured over a time interval
- Applications: Stock price prediction, scientific simulation, health analytic

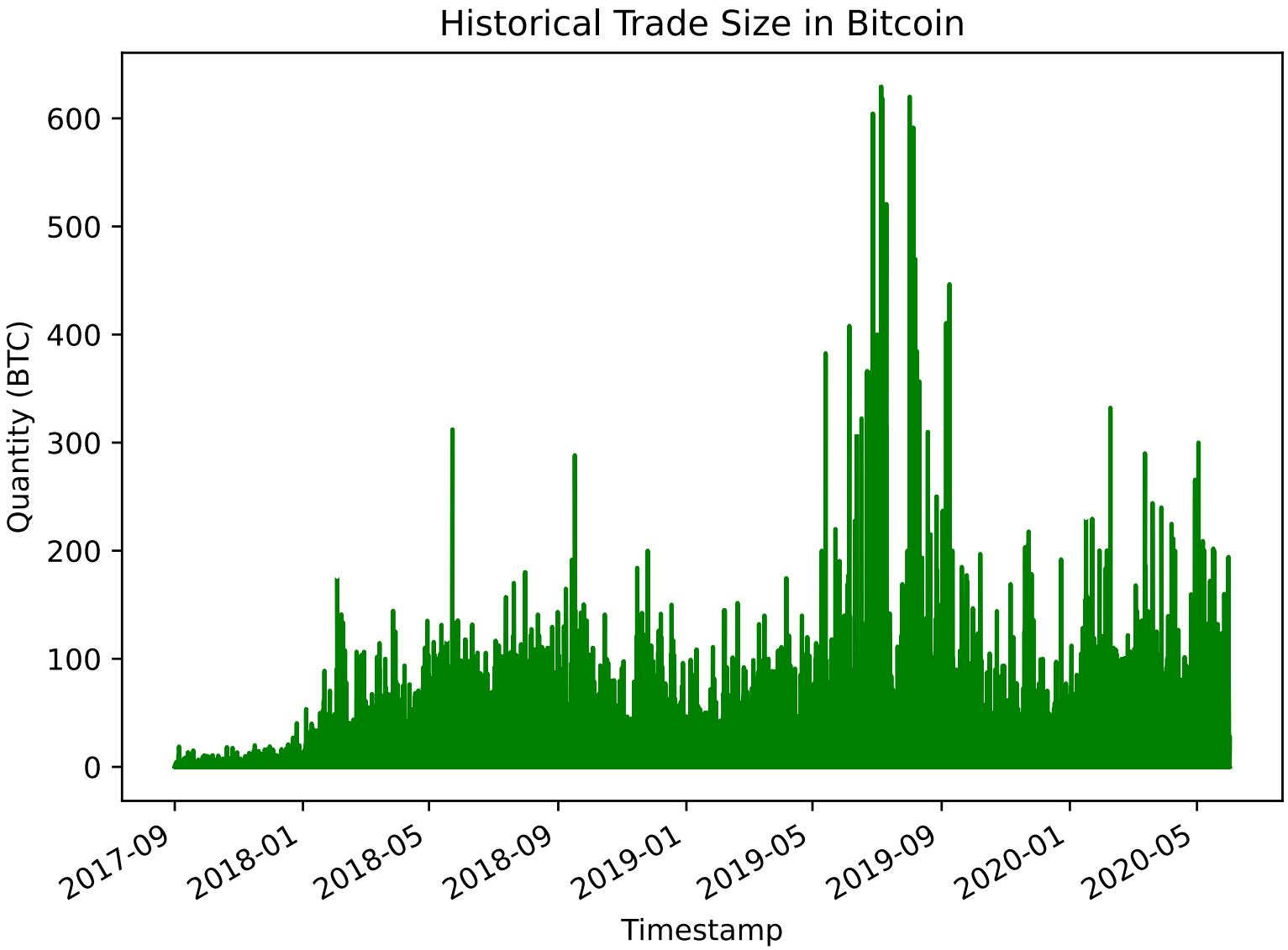


Sample of a time-series table

BTCUSDT Historical Tick Data

Timestamp	Price($)	Qty (BTC)	Trade ID	First tradeID	Last tradeID	Buyer marker	...
2017-09-01 00:01:00.493	4689.89	0.053417	61458	69180	69180	True	...
2017-09-01 00:01:00.530	4689.89	0.064732	61459	69181	69182	True	...
2017-09-01 00:01:00.573	4689.90	0.147065	61460	69183	69183	True	...
2017-09-01 00:01:18.120	4689.90	0.042047	61461	69184	69184	True	...
2017-09-01 00:01:18.210	4689.91	0.115178	61462	69185	69185	True	...
...	...	...	...	...	...	...	...

Sample of a time-series chart



Research Problems

- Current approach to dealing with time-series data does not involve an exploitation of its structure
- Maintenance of *B-Tree*/B^+-*Tree* structures requires sizable amount of memory to maintains an $O(log\ n)$ complexity
- Compression and/or hashing are computationally expensive
- Often times, current approach only supports **seconds** or **milliseconds** granularity due to the limitation of *B-Tree*
- Non-persistent coupled with limited scalability due to memory constraint and inability to fully utilize HDD/SSD
- Querying relies on index. Impractical for fine-grained dataset
  1 year data = $31,536,000,000$ (ms) $\approx$ 31.5 billion indexes







Order Magnitude of Time

1-Year worth of data

Interval	No. of entries
1-Month	12
1-Week	52
1-Day	365
1-Hour	8,760
1-Minute	525,960
1-Second	31,536,000
1-Millisecond	31,536,000,000
1-Microsecond	31,536,000,000,000
1-Nanosecond	31,536,000,000,000,000

1 Introduction

2 XStore

3 Evaluations

4 Next Steps

Introducing XStore

XStore is a distributed *NoSQL* database program designed to address an ever-growing amount of time-series. Equipped with technique that exploits an order of magnitude of time to deliver an extreme performance

Why XStore:

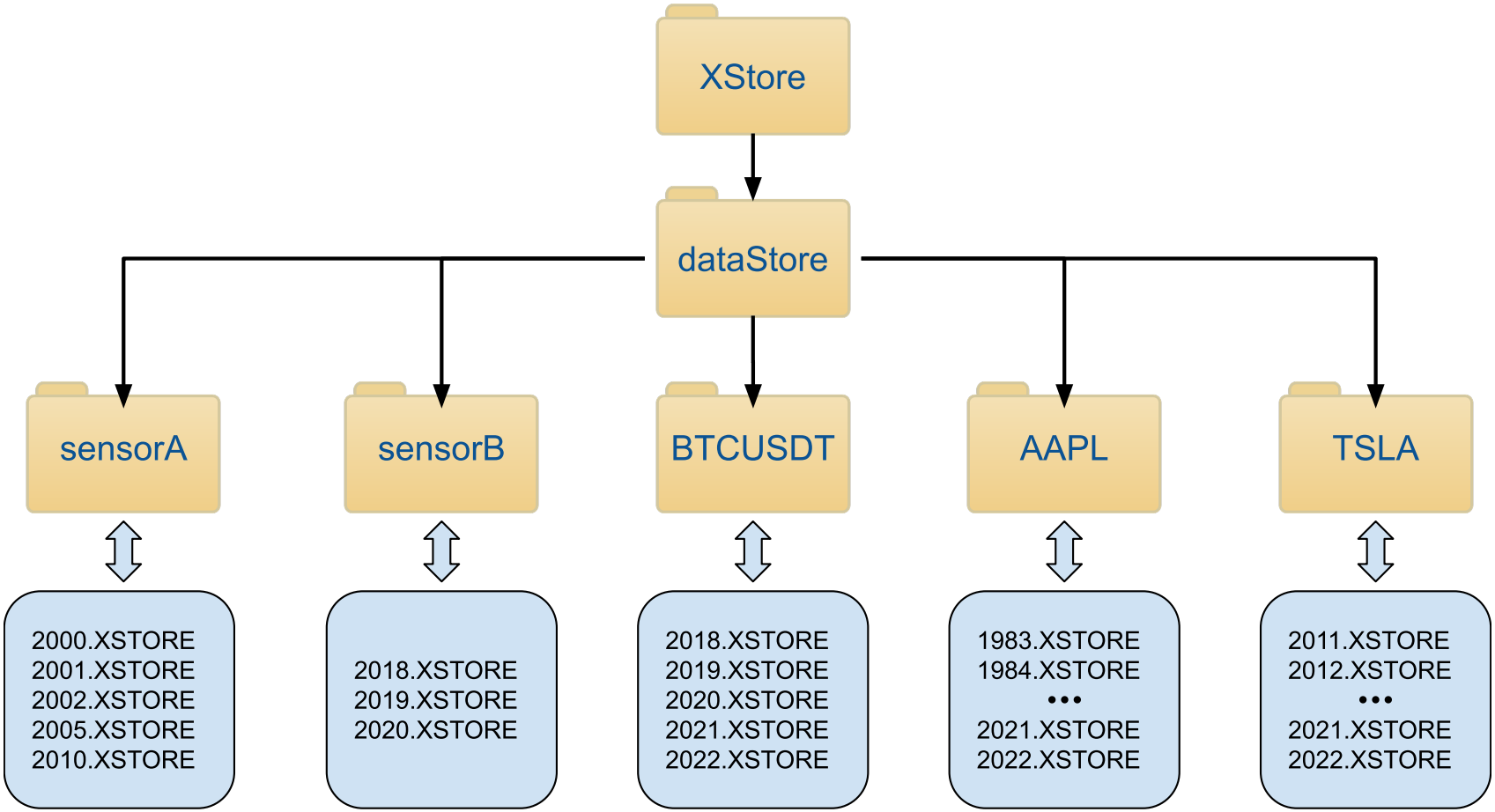
- Guarantees $O(1)$ complexity for insert, get and remove operations
- Does not maintain hash-map, reduces processor and memory costs
- Eliminates the needs of hashing through time structure exploitation
- Remains scalable regardless of granularity and size of time-series data
- Fault-tolerant due to "Stores nothing in memory, always on disk" policy



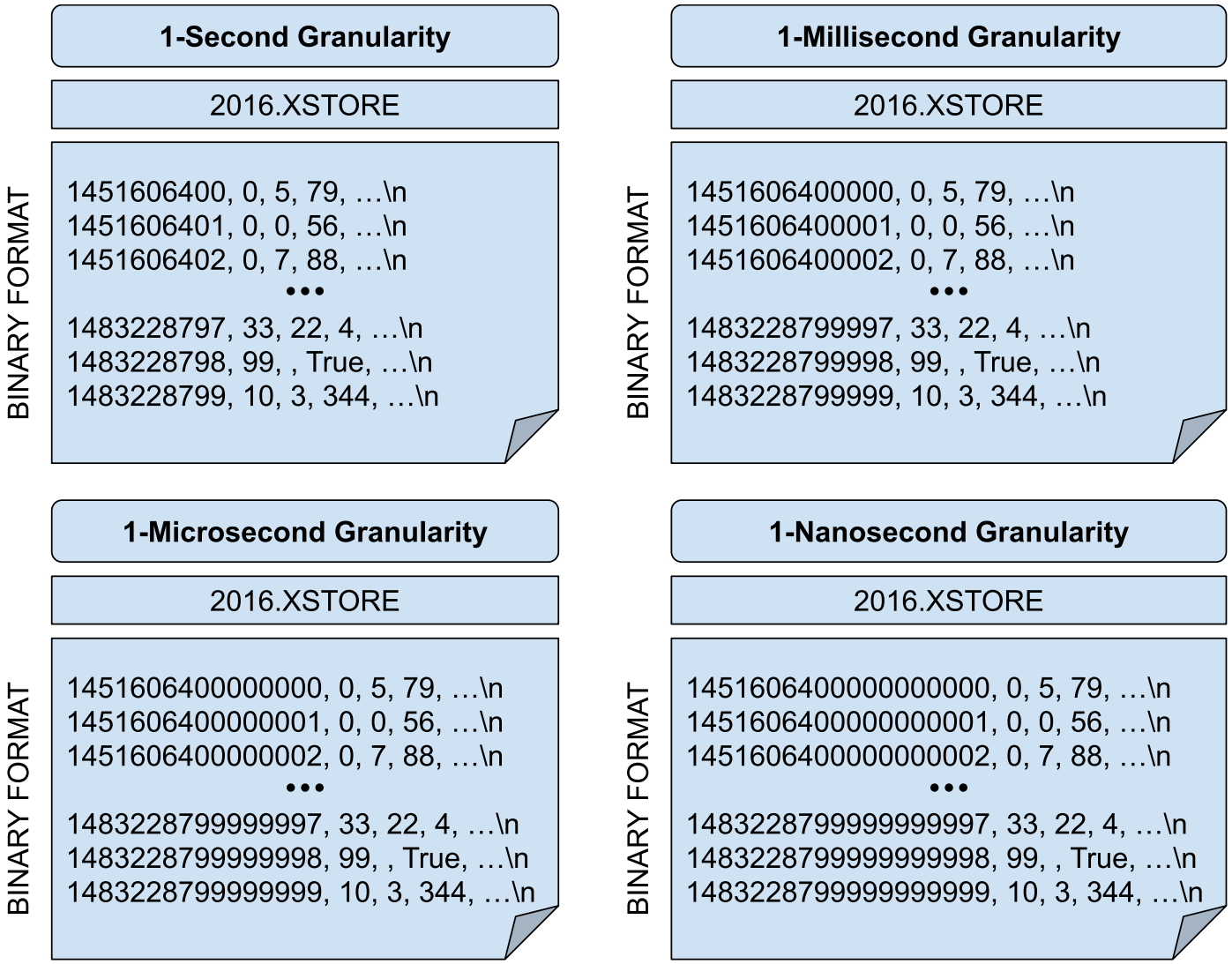




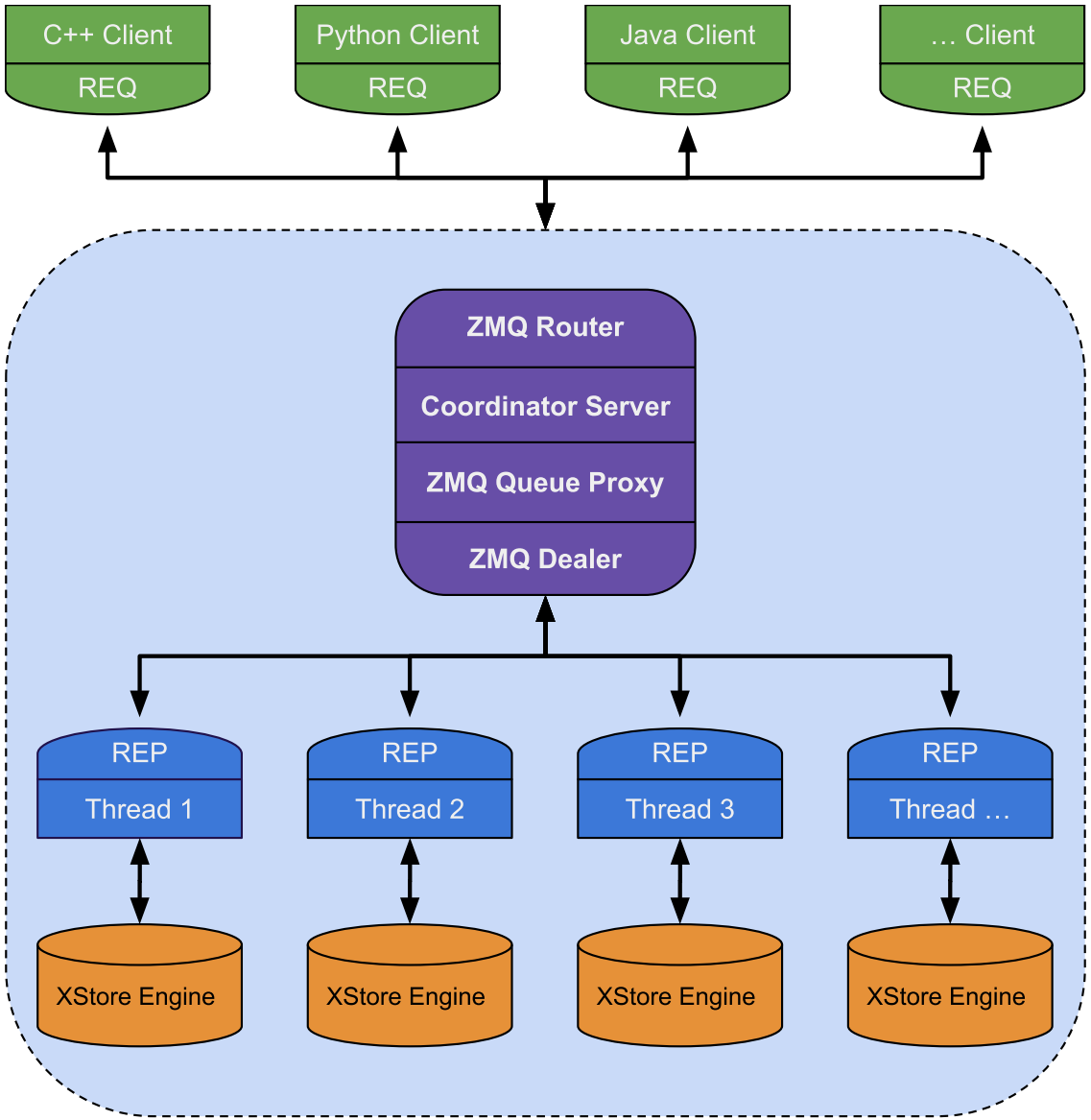
File Organization



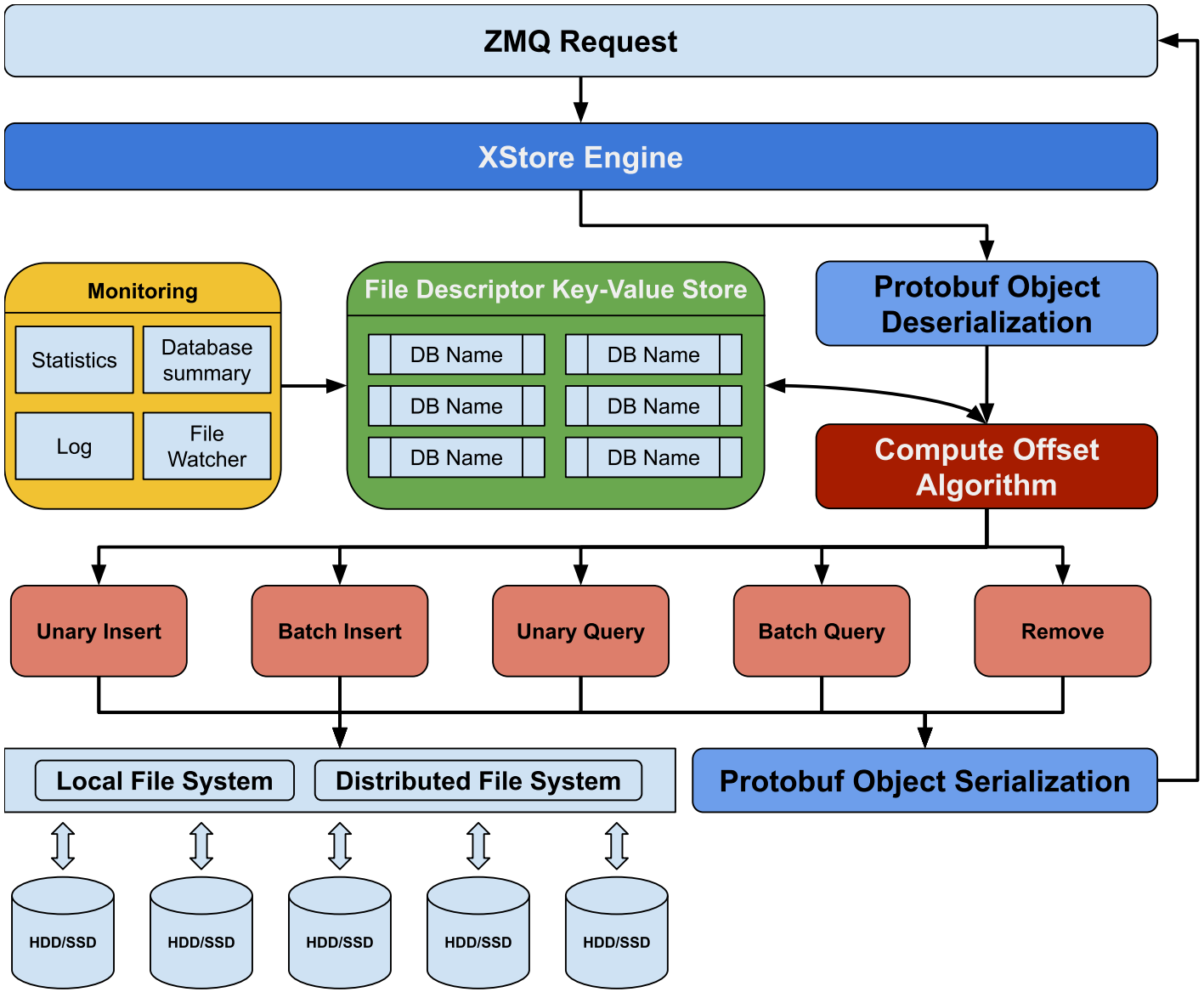
Data Organization



Outer Layer – Networking



Inner Layer – XStore Engine



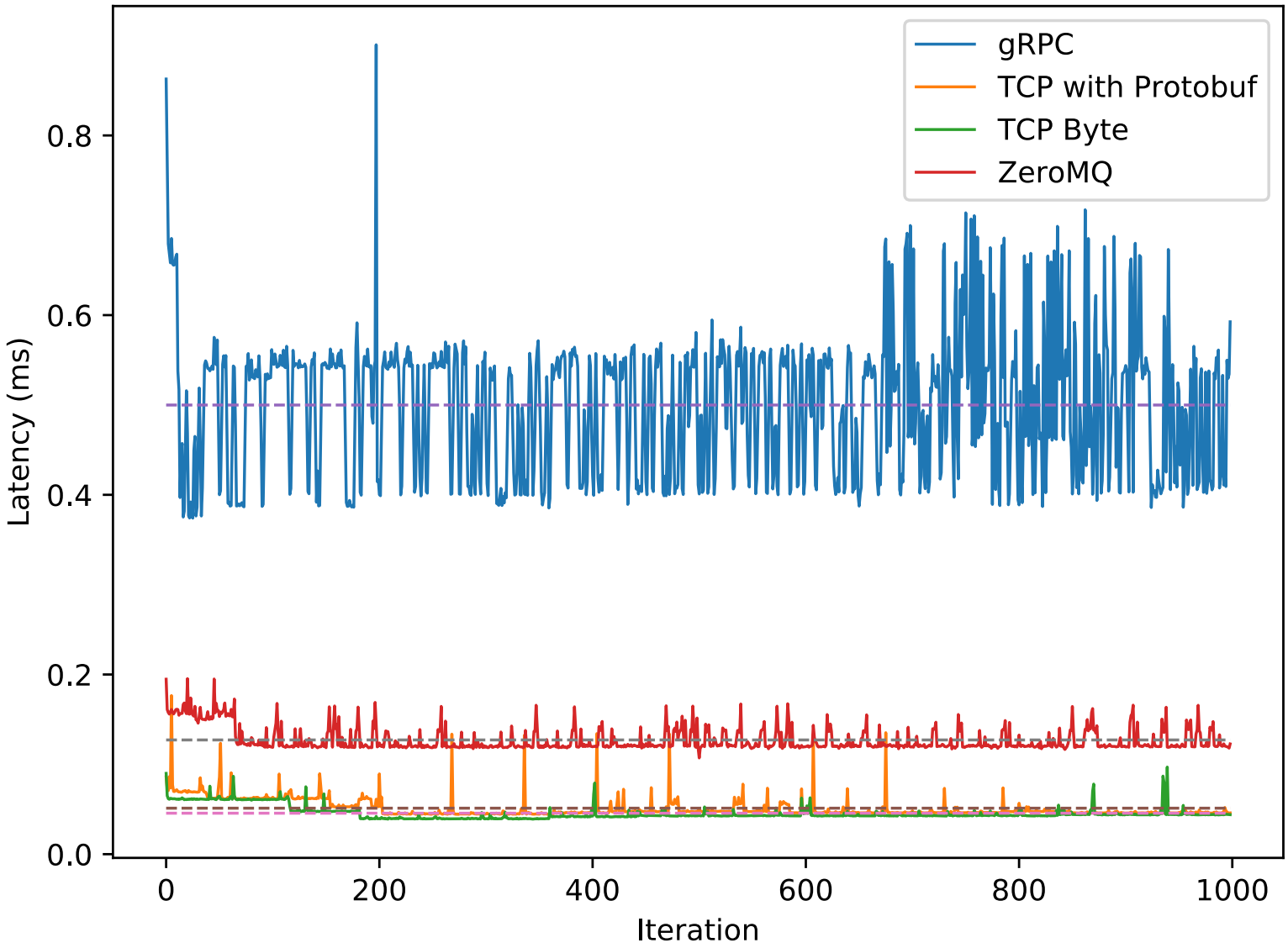
1 Introduction

2 XStore

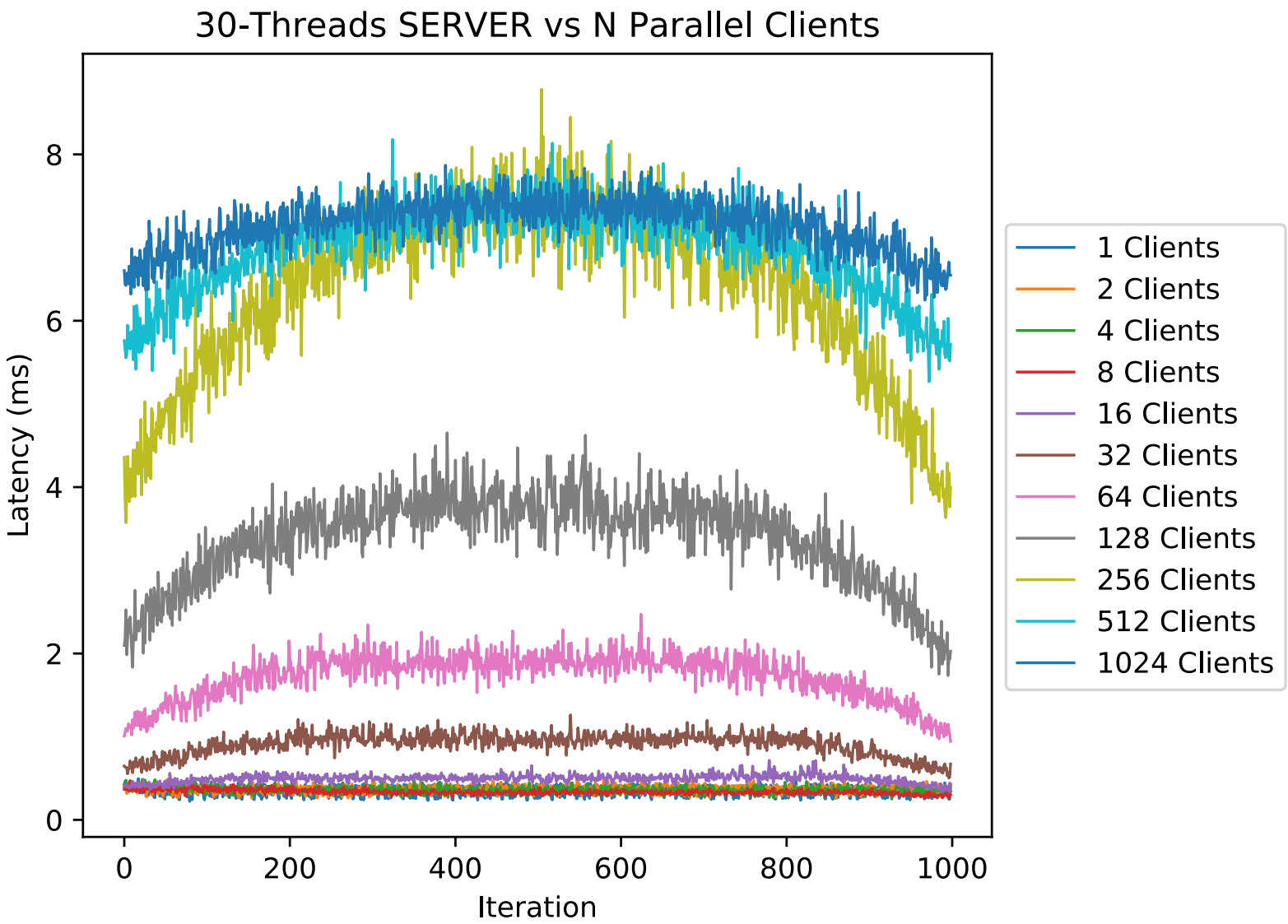
3 Evaluations

4 Next Steps

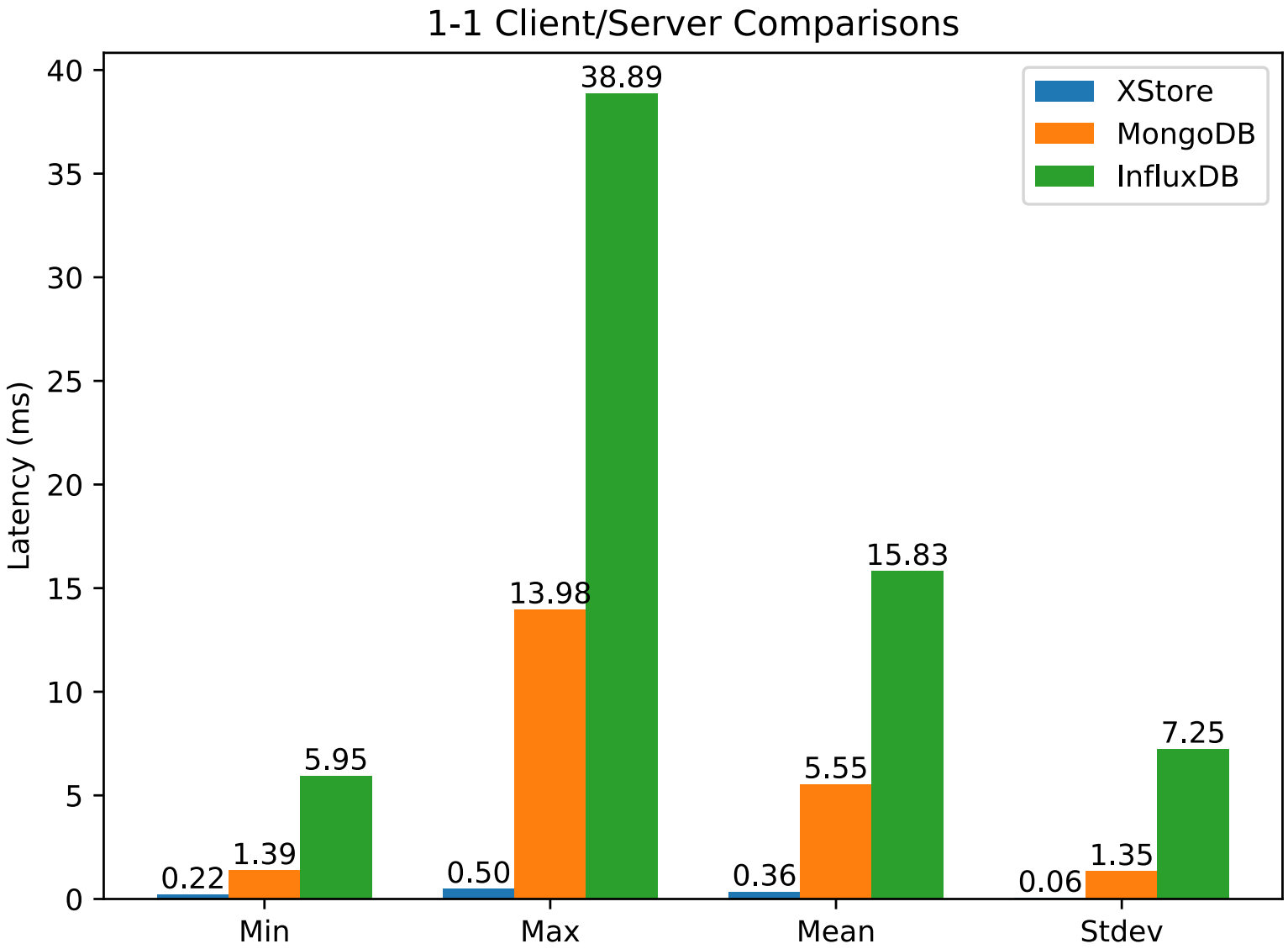
Microbenchmark – Latency



Microbenchmark – Unary Query



Microbenchmark – Unary Query (Quick comparison)



1 Introduction

2 XStore

3 Evaluations

4 Next Steps

Next Steps – Exciting journey ahead

1. Find papers employing this technique
2. More benching – Bench on 2y, 4y, 8y, 10y+ on crypto dataset
   To prove XStore is scalable regardless of data size
3. Bench on PCIe NVMe SSD & IOzone
4. Potentially utilize memory to speed-up
5. Research low-latency distributed filesystem
   Perhaps, another exciting research topic?
6. Branch-out XStore's techniques to *Pandas – Python*
7. More functionality:
   - Python library to consume XStore
   - Developing insert/get batch operations
   - Enhance current XStore's monitoring & log operations
   - Performance mode: Speed/Space
   - Implement binary search for Space Mode
   - Support Intel® Optane mode

Thank you for your time and patience!